



Manual Técnico

Proyecto: CONTROL EAT FOOD

DESARROLLO DE SOFTWARE

Universidad Central del Ecuador

Facultad de Ingeniería y Ciencias Aplicadas

Sistemas de Información



CONTROL EAT FOOD

MANUAL TÉCNICO

CONTENIDO

CONTENIDO	3
1. Objetivos	3
1.1 Objetivos Específicos.	4
2. Alcance	4
3. Requerimientos Técnicos	4
3.1 Requerimientos Mínimos de Hardware.	4
3.2 Requerimientos Mínimos de Software.	5
4. Herramientas Utilizadas para el Desarrollo.	5
5. Instalación	5
6. Configuración	6
7. Diseño de la Arquitectura Física.	8
8. Usuarios	8
8.1 Usuarios de base de datos	9
8.2 Usuarios de sistema operativo	9
8.3 Usuarios de aplicaciones	10
9. Contingencias y soluciones.	10

1. Objetivos

Este documento tiene como propósito documentar el diseño, la instalación, la configuración, la operación y el mantenimiento del sistema Control Eat Food, una solución empresarial para el control del consumo de alimentos del personal mediante identificación biométrica por huella digital (lector ZKTeco ZK9500 con identificación 1:N).

El sistema se compone de un backend REST desarrollado en Spring Boot, un frontend web tipo PWA en React y una aplicación móvil Android, integrados con una base de datos MySQL.

- 1.1 **Objetivos Específicos.**
- Detallar los requerimientos técnicos necesarios para la ejecución del sistema.
 - Documentar los procesos de instalación y configuración del backend, el frontend web y la app móvil.
 - Especificar la arquitectura física y lógica del sistema y su integración biométrica.
 - Describir los perfiles de usuario (roles) del sistema y sus medidas de seguridad.
 - Prever posibles fallas y documentar sus soluciones.

2. **Alcance**

El documento describe la versión estable de Control Eat Food, desplegada en un entorno cliente-servidor: un backend Spring Boot con base de datos MySQL, un frontend web PWA (React) y una aplicación móvil Android, comunicados mediante una API REST sobre HTTP/HTTPS.

Este documento está dirigido a:

- Administradores del sistema y personal de TI.
- Técnicos de soporte e implementación.
- Personal responsable del despliegue y mantenimiento del servidor.
- Personal de catering/restaurante encargado de la operación de los kioscos.

Conocimientos básicos en:

- Administración de servidores (Windows/Linux).
- Java 21 y Spring Boot (backend).
- Bases de datos MySQL.
- Redes, HTTP/HTTPS y despliegue (Nginx opcional; el instalador Inicio.bat configura todo automáticamente).
- Instalación de dependencias (Maven, Node.js/npm y Gradle).

3. **Requerimientos Técnicos**

3.1 **Requerimientos Mínimos de Hardware.**

Recurso	Valor mínimo recomendado
Procesador	Intel Core i5 / Xeon 2.4 GHz o equivalente AMD
Memoria RAM	8 GB (backend Spring Boot + base de datos MySQL)

Espacio en Disco	2 GB libres (aplicación, base de datos y respaldos)
-------------------------	---

3.2 Requerimientos Mínimos de Software.

Componente	Requerido
Sistema Operativo	Servidor: Windows 10/11, Clientes: navegador moderno (web) y Android 10+ (móvil)

Privilegios de Administrador: Sí (para instalar Java, Node.js, MySQL/Docker y el agente del lector ZK9500)

4. Herramientas Utilizadas para el Desarrollo.

1. Java 21 + Spring Boot 3

Descripción: Plataforma del backend que expone la API REST, gestiona la seguridad (JWT), la lógica de negocio y el motor de identificación biométrica 1:N.

Motivo por el cual fue utilizado: Robustez, ecosistema maduro, sólido soporte de seguridad y rendimiento adecuado para una aplicación empresarial.

2. React + Vite (PWA)

Descripción: Framework del frontend web utilizado para el panel de administración, el modo kiosco de restaurante y el soporte offline (IndexedDB).

Motivo por el cual fue utilizado: Permite un desarrollo ágil de interfaces reactivas, capacidades PWA (instalable y offline) y buen rendimiento en el navegador.

3. Kotlin + Jetpack Compose (Android)

Descripción: Tecnología de la aplicación móvil para la gestión administrativa y el modo kiosco, con lectura biométrica por USB-OTG y almacenamiento offline mediante Room.

Motivo por el cual fue utilizado: UI declarativa moderna, integración nativa con el hardware del dispositivo y soporte oficial de Android.

4. MySQL 8 + Flyway

Descripción: Sistema de base de datos relacional que almacena empleados, consumos, huellas cifradas, usuarios y auditoría. Flyway administra las migraciones del esquema.

Motivo por el cual fue utilizado: Fiabilidad, amplio soporte y control de versiones del esquema mediante migraciones reproducibles.

5. Lector ZKTeco ZK9500 + SDK ZKFinger

Descripción: Hardware y SDK de captura de huella. En la web se comunica mediante un agente local por WebSocket (ZKFinger WebAPI) y en Android mediante USB-OTG.

Motivo por el cual fue utilizado: Compatibilidad con la identificación 1:N y disponibilidad de SDK para integrarlo en ambos clientes. IDEs empleados: IntelliJ IDEA, Visual Studio Code y Android Studio.

5. Instalación

5.1 Requisitos Previos

- Tener instalado Java JDK 21, Node.js (LTS) y MySQL 8 (local o en contenedor Docker).
- Instalar el agente/SDK del lector ZK9500 para la comunicación por WebSocket (RunWindows/setup.exe).

En Windows, hacer doble clic en RunWindows/Inicio.bat para lanzar el instalador unificado. El script solicita permisos de Administrador (UAC) automáticamente y presenta un menú interactivo con opciones de: [1] Pruebas (desarrollo), [2] Instalación nueva (Producción), [3] Actualizar, [4] Reparar, [5] Desinstalar, [6] Diagnósticos, [7] Salir. El script verifica e instala automáticamente Java 21, Node.js y Maven vía winget.

5.2 Instalación del Sistema

- Clonar o descargar el repositorio del proyecto (proyectoFinal). Navegar a la carpeta RunWindows/.
- Ejecutar Inicio.bat y seleccionar la opción deseada: [1] Pruebas abre el backend (mvn spring-boot:run) y frontend (npm run dev) en ventanas separadas; [2] Instalación nueva compila todo, integra el frontend en el backend y registra el servicio de Windows con NSSM. Para la app móvil, compilar el APK con Gradle (./gradlew :app:assembleRelease).

5.3 Comprobar la estructura:

proyectoFinal/

```

├── controlEatFoodWeb/
|   ├── backend/      (Spring Boot · API REST · biometría 1:N)
|   |   ├── src/main/java/com/eatfood/control/
|   |   └── src/main/resources/ (application.yml, db/, images/)
|   └── frontend/     (React + Vite · PWA · modo kiosco)
|       ├── src/
|       └── public/
├── controlEatFoodMovil/ (App Android · Kotlin + Jetpack Compose)
|   └── app/src/main/
├── RunWindows/      (instalador unificado: Inicio.bat + Inicio.ps1)
├── .gitignore
└── README.md

```

6. Configuración

La configuración del backend se realiza mediante el archivo application.yml y variables de entorno de seguridad. No se almacenan credenciales en el código fuente: las claves sensibles (base de datos, JWT y cifrado biométrico) se inyectan como variables de entorno.

6.1 Parámetros configurables (variables de entorno)

Parámetro	Origen	Ejemplo	Descripción
DB_URL	Variable de entorno / application.yml	jdbc:mysql://localhost:3306/control_eat_food	Cadena de conexión a la base de datos MySQL
DB_USER / DB_PASSWORD	Variable de entorno	control / *****	Credenciales de acceso a la base de datos
JWT_SECRET	Variable de entorno	Clave Base64 de 256 bits	Clave para firmar y validar los tokens JWT
BIOMETRIC_ENCRYPTION_KEY	Variable de entorno	Clave AES de 16 bytes	Clave AES-128 para cifrar las plantillas de huella

6.2 Endpoints y operaciones clave de la API REST

El sistema expone un conjunto de servicios estructurados en módulos jerárquicos protegidos mediante Spring Security. Las rutas de la API operan bajo el prefijo unificado `/api` y requieren de un token JWT válido (a excepción de las rutas de autenticación pública):

MÓDULO DE AUTENTICACIÓN Y SEGURIDAD

- **POST /api/auth/login:** Recibe las credenciales de usuario, valida mediante hash BCrypt y retorna el token de acceso JWT y el refresh token.
- **POST /api/auth/refresh:** Genera un nuevo token JWT de acceso sin solicitar credenciales adicionales, utilizando el refresh token almacenado de forma segura.
- **POST /api/auth/logout:** Invalida el token de acceso actual en el servidor y cierra la sesión de forma segura.

MÓDULO DE CONSUMOS Y BIOMETRÍA (CORE)

- **POST /api/scan:** Endpoint principal del sistema. Recibe la plantilla de minucias en Base64, ejecuta la identificación 1:N empleando el motor nativo de ZKTeco, valida las reglas de negocio (horarios autorizados, control de consumos duplicados) y registra el consumo en caso de éxito.
- **POST /api/manual-consumptions:** Registra un consumo forzado de forma manual (exclusivo para rol ADMIN) omitiendo la validación biométrica en casos excepcionales.
- **POST /api/manual-consumptions/external:** Registra un consumo asignado a visitantes o contratistas externos mediante su número de cédula, creando de forma automatizada un empleado temporal de carácter inactivo.
- **POST /api/scan/sync:** Recibe arreglos estructurados de consumos almacenados localmente durante fallas de conectividad para su inserción masiva y sincronización en la base de datos central.
- **GET /api/scan/today:** Recupera en tiempo real el listado de personas registradas durante el día actual en un punto de catering específico para poblar el panel dinámico del kiosco.

MÓDULO DE REPORTES, AUDITORÍA Y DASHBOARD

- **GET /api/reports/dashboard:** Entrega los indicadores clave de rendimiento (KPIs) y el resumen cuantitativo de consumos del día en curso.
- **GET /api/reports/trend:** Provee la matriz de datos de consumo de los últimos 7 días con el fin de renderizar las gráficas analíticas de

tendencias en las interfaces.

- **GET /api/reports/consumptions:** Consulta el histórico detallado de consumos aplicando filtros avanzados por rangos de fecha y puntos de catering.
- **GET /api/reports/export?format=pdf|excel|csv:** Genera y transmite binarios de reportes formateados con firmas digitales corporativas, logos de la institución y marcas de agua.
- **GET /api/audit:** Extrae las trazas del log de auditoría inmutable reflejando las operaciones críticas realizadas en la plataforma.

MÓDULO DE MANTENIMIENTO DE DATOS

- **GET, POST, PUT, DELETE /api/employees:** Operaciones CRUD completas para la gestión de datos demográficos y estados de enrolamiento de empleados (el borrado físico se mitiga mediante un borrado lógico/inactivación).
- **GET /api/fingerprints/employee/{id}:** Recupera los metadatos de las huellas digitales asociadas a un legajo de empleado.
- **POST /api/fingerprints/enroll:** Registra y persiste una plantilla de huella digital procesada en la base de datos central.
- **DELETE /api/fingerprints/{id}:** Remueve una huella específica del repositorio biométrico.
- **GET, POST, PUT /api/positions:** Administración de cargos y estructuras organizacionales.
- **GET, POST, PUT /api/caterings:** Gestión física de los puntos de distribución de alimentos y kioscos autorizados.
- **GET /api/meal-types:** Lista los tipos de servicios definidos (ej. Almuerzo, Merienda).
- **GET, POST /api/schedules:** Asigna y delimita las ventanas horarias operativas para cada tipo de comida.

7. Diseño de la Arquitectura Física.

Elemento	Descripción
Nombre de los equipos	Servidor de aplicación (backend Spring Boot) y servidor de base de datos MySQL; pueden ser el mismo equipo
Ubicación física	Servidor local de la institución o instancia en la nube; kioscos de restaurante y dispositivos Android en cada punto de consumo
Direcciones IP	Backend accesible por HTTP/HTTPS en la red local o mediante dominio público (PUBLIC_URL)
Puertos TCP/UDP	3000 (producción, configurable) / 8080 (desarrollo) para API REST + WebSocket, 3306 (MySQL). En producción con Inicio.bat, el frontend se integra en el backend (carpeta static/) y Nginx es opcional
Dependencias externas	MySQL 8, lector ZKTeco ZK9500 y su agente/SDK (WebSocket en web, USB-OTG en Android)
Interrelaciones	Lector ZK9500 ↔ Cliente (web/móvil) ↔ API REST (Spring Boot) ↔ MySQL

8. Usuarios

El sistema implementa autenticación con tokens JWT y control de acceso por roles. Existen tres roles de seguridad: Administrador, Supervisor y Restaurante (rol interno CATERING). A continuación se clasifican los usuarios según el contexto técnico y de

USO:

8.1 Usuarios de base de datos

El sistema utiliza una base de datos MySQL (control_eat_food). El backend accede a ella mediante una cuenta de servicio dedicada; las plantillas de huella se almacenan cifradas y nunca se exportan.

Atributo	Valor
Nombre del usuario	control (usuario de la base de datos MySQL)
Descripción/Propósito	Cuenta de servicio usada por el backend para acceder a la base control_eat_food
Grupos a los que pertenece	N/A (usuario dedicado de la aplicación)
Privilegios generales (DB)	SELECT, INSERT, UPDATE, DELETE sobre la base control_eat_food
Privilegios sobre objetos	Gestión de tablas de empleados, consumos, huellas (cifradas), usuarios y auditoría; migraciones con Flyway

Las plantillas biométricas se almacenan cifradas (AES-128/CBC) y nunca se exponen. El acceso a la base de datos está restringido al backend, que valida cada petición con JWT y con el rol del usuario.

Seguridad Criptográfica y Mitigación de Riesgos Biométricos

Para cumplir con los estándares de protección de datos de carácter personal y biometría, el sistema implementa capas estrictas de seguridad de la información:

1. **Cifrado de Plantillas Biométricas (AES-128/CBC):** Las minucias capturadas por el lector ZK9500 en ningún escenario se almacenan en texto plano ni se guardan imágenes de las huellas dactilares. El backend toma la cadena binaria y aplica un algoritmo simétrico de cifrado AES (Advanced Encryption Standard) con bloques de 128 bits en modo CBC (Cipher Block Chaining), utilizando un vector de inicialización (IV) aleatorio y la clave persistida en la variable de entorno 'BIOMETRIC_ENCRYPTION_KEY'. Si la base de datos MySQL sufriera un acceso no autorizado, los registros de huellas resultan inútiles para la reconstrucción de identidades.
2. **Protección de Credenciales:** El motor de autenticación emplea la función de derivación de claves adaptativa BCrypt para el hashing de contraseñas de los usuarios de la aplicación, implementando un factor de trabajo que ralentiza ataques de fuerza bruta por hardware.
3. **Bloqueo Anti-Fuerza Bruta:** El backend acopla un interceptor de seguridad que contabiliza los intentos erróneos de inicio de sesión de forma consecutiva. Al registrarse el quinto (5to) intento fallido sobre un mismo usuario, la cuenta es suspendida automáticamente por un periodo de tiempo configurable para frustrar ataques automatizados de diccionario.
4. **Auditoría Inmutable:** Los eventos críticos del sistema (creación, modificación, borrado lógico, autorizaciones manuales y descargas de reportes) se graban de manera síncrona en una tabla dedicada de auditoría. El diseño del módulo restringe los privilegios de actualización (UPDATE) o eliminación (DELETE) sobre esta tabla para todos los roles del aplicativo, garantizando la trazabilidad no repudiable ante procesos de fiscalización institucional.

8.2 Usuarios de sistema operativo

Este usuario representa a la persona que despliega y mantiene el sistema en el servidor. Requiere privilegios para instalar dependencias, administrar el servicio y configurar el proxy inverso.

Atributo	Valor
----------	-------

Nombre del usuario	Administrador del servidor
Dominio	Servidor de la institución (local o en la nube)
Descripción/Propósito	Despliega y mantiene el backend, la base de datos y el frontend; gestiona el servicio de Windows (NSSM vía Inicio.bat) o systemd en Linux
Grupos a los que pertenece	Administradores / sudoers
Privilegios sobre carpetas	Lectura/escritura sobre la carpeta de la aplicación, logs, respaldos de MySQL y certificados TLS

Se recomienda ejecutar el backend con un usuario de servicio dedicado y privilegios mínimos, reservando la cuenta de administrador solo para instalación y mantenimiento.

8.3 Usuarios de aplicaciones

Este es el usuario que interactúa directamente con el sistema desde la web o la app móvil. Sus acciones dependen del rol asignado (Administrador, Supervisor o Restaurante).

Atributo	Valor
Nombre del usuario	Administrador · Supervisor · Restaurante (Catering)
Descripción/Propósito	Operan el sistema desde la web o la app móvil según su rol
Grupos a los que pertenece	ADMIN, SUPERVISOR, CATERING (roles de seguridad)
Privilegios dentro de la aplicación	ADMIN: gestión total (empleados, usuarios, horarios, reportes y auditoría) SUPERVISOR: consulta de dashboard y reportes RESTAURANTE (CATERING): modo kiosco, registro de consumos por huella y reporte diario

Cada rol accede únicamente a las funciones autorizadas. Todas las acciones críticas quedan registradas en un log de auditoría inmutable.

9. Contingencias y soluciones.

A continuación se describen las posibles contingencias que pueden surgir durante la instalación, ejecución y uso de Control Eat Food, así como sus soluciones recomendadas para mitigar o corregir los incidentes.

Nº	Contingencia	Causa Probable	Solución Recomendada
1	El backend no arranca (mvn/JAR falla)	Java 21 no instalado o variables de entorno de seguridad sin definir	Instalar el JDK 21 y definir DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET y BIOMETRIC_ENCRYPTION_KEY antes de iniciar.
2	Error de conexión a la base de datos	MySQL apagado o credenciales/URL incorrectas	Verificar que el servicio/contenedor MySQL esté activo y que DB_URL, usuario y clave sean correctos. Revisar el puerto 3306.
3	El lector ZK9500 no es detectado en la web	Agente/SDK ZKFinger no instalado o WebSocket	Instalar el agente (RunWindows/setup.exe) y permitir la conexión al WebSocket /zkfinger-ws. Reconectar el lector por USB.

		bloqueado	
4	El lector no funciona en la app móvil	Permiso USB-OTG denegado o SDK ausente	Conceder el permiso USB al conectar el lector y verificar que el .aar del SDK esté en app/libs/.
5	Sesión caducada / error 401 al usar la API	Token JWT expirado	Volver a iniciar sesión. El cliente renueva el token automáticamente mediante el refresh token.
6	Los consumos no se registran sin conexión	<i>Pérdida temporal de enlace WAN o desconexión del servidor de base de datos.</i>	El sistema encola los registros offline (IndexedDB en web, Room en móvil) y los sincroniza al restablecerse la conexión.
7	El puerto del backend (3000 en producción / 8080 en desarrollo) está ocupado	Otro proceso utiliza el puerto	Detener el proceso en conflicto o cambiar server.port en application.yml.
8	La huella no identifica al empleado	Empleado sin huella registrada o captura de baja calidad	Registrar o actualizar la huella del empleado desde el módulo de Empleados y repetir la captura.
9	El reporte PDF/Excel no se descarga	Bloqueo del navegador o falta de permisos de rol	Permitir las descargas en el navegador y verificar que el usuario tenga rol ADMIN para exportar.

10. Checklist de Integridad del Repositorio para el Despliegue en Producción

Previo al empaquetamiento final y defensa del software, el responsable técnico debe constatar el cumplimiento estricto del siguiente checklist estructural dentro del árbol del repositorio 'proyectoFinal' en GitHub para garantizar la portabilidad e integridad del sistema:

- **Repositorio de Librerías Nativas Android:** Constar que el archivo binario empaquetado del SDK biométrico de ZKTeco ('SDK ZKFinger') se encuentre correctamente ubicado en la ruta relativa 'controlEatFoodMovil/app/libs/'. La ausencia física de este recurso o de los binarios '.so'/' .jar' impedirá la compilación exitosa del APK móvil mediante Gradle en entornos de integración continua o despliegues limpios.
- **Aislamiento de Variables de Entorno en Scripts de Windows/Linux:** Verificar que la carpeta 'RunWindows/' contenga el instalador unificado: 'Inicio.bat' (punto de entrada con elevación UAC) e 'Inicio.ps1' (script de ~1500 líneas que gestiona Pruebas, Producción, Actualización, Reparación, Desinstalación y Diagnósticos). Queda estrictamente prohibido persistir valores de producción reales o cadenas de conexión explícitas (claves DB, credenciales raíz de MySQL, o llaves privadas de firmas 'JWT_SECRET') dentro del repositorio público de GitHub. El script Inicio.ps1 solicita estas variables de forma interactiva al operador durante la instalación y las persiste de forma segura en 'RunWindows/config/' (excluido del repositorio por .gitignore) o las inyecta como variables de entorno del servicio NSSM.

- **Integridad de Migraciones Flyway:** Asegurar que los scripts SQL estructurados dentro de 'backend/src/main/resources/db/migration/' mantengan una numeración secuencial estricta e inmutable. El hash de verificación calculado por Flyway no debe ser alterado de forma manual para evitar fallas de inconsistencia de esquema en el arranque de la API REST de Spring Boot.

ACTUALIZACIÓN: DETALLES TÉCNICOS Y ARQUITECTURA

1. Arquitectura del Sistema

El sistema emplea una arquitectura cliente-servidor compuesta por tres partes:

- Backend (Spring Boot 3): API REST, lógica de negocio y procesamiento biométrico 1:N.
- Frontend (React + Vite PWA): Panel web administrativo y Kiosco.
- Móvil (Android/Kotlin): App con Jetpack Compose y uso del SDK biométrico nativo por USB OTG.

2. Tecnologías Principales

- Backend: Java 21, Spring Boot, Spring Security (JWT), Hibernate/JPA, Flyway.
- Base de Datos: MySQL (8.0+).
- Frontend Web: Node.js 18+, React, Vite, IndexedDB (Cola Offline).
- Frontend Móvil: Kotlin, Jetpack Compose, Retrofit, Room (Cola Offline).
- Biometría: SDK nativo ZKTeco (ZK9500), ZKFinger WebAPI vía WebSocket.

3. Seguridad y Cifrado

- Autenticación JWT: Endpoints asegurados por roles y bloqueo anti-fuerza bruta tras 5 intentos fallidos.
- Privacidad Biométrica (AES-128): Las plantillas de huellas se cifran usando AES-128 modo CBC (BIOMETRIC_ENCRYPTION_KEY).
- Auditoría: Transacciones críticas se registran en la tabla inmutable AuditLog.

4. Sincronización Offline y Tolerancia a Fallos

- Los clientes (Web/Móvil) encolan localmente los consumos cuando la red cae.
- Al restaurar la conexión, se envían por lotes a /api/scan/sync.
- Idempotencia: El backend garantiza la no duplicación gracias al índice único por empleado, día y tipo de comida.